MMA3001

Project Report

Project Scope

Develop a computer-vision method that analyses images of packaged pork rashers to detect and localise packaging or product-placement defects.

Robert D'Assisi
34988998

Engineering Problem

Context: The engineering context is a meat processing production line packaging pork rashers into trays before distribution. The defects that can occur when being packaged include unsealed or wrinkled film, loose meat or twisted rashers inside the tray. Normally, human inspection would occur to supervise trays as they pass which is repetitive, subjective and sustaining accuracy at a production line speed isn't ideal. A computer vision inspection will reduce reliance and aid manual detection and be efficient enough to keep up with production line pace and catch defects before external distribution.

Problem: To address the problem the dataset which included (Roboflow "Pork Rasher Error (Packaging)", v4, 3,549 images, human drawn boxes around defects in five classes) is built for object detection. The narrower problem chosen was measuring the severity or extent of defects and turning it into a pass/fail decision. To predict this a tray's defect area fraction from a single photo was compared to a threshold to decide a pass or fail. The target is computed from the given label files and used only to train and score, at prediction the model only saw the image without its label.

Why It Matters: The problem matters because a single pass/fail only shows the presence of a defect without measuring its severity. Measuring severity allows production lines to prioritise certain defected trays and gives a measurable quantity to track inspection performance. It tests the major question that is a simple, fast method accurate and quick enough for production line use?

Intended Use: This computer vision inspection solution is intending to be a screening aid not a full replacement of inspectors. Its input is a 720 x 540 RGB image (.jpg, .jpeg, .png). Its output is a predicted defect area fraction, pass/fail decision and the threshold used (default 0.04 from validation split). Invalid inputs were rejected with an error and never given a prediction. The method uses regression models taught in the unit (linear regression, decision tree regression and SVR) on hand crafted features such as a HSV colour histogram and finite difference edge strength.

Limitations: The limitation of the selected scope is the lack of localisation, the model only estimates how much of a defect there is without knowing where it is, so it isn't a detector. The target is a proxy as boxes that overlap count as double and due to image cropping from roboflow not all the package tray is in frame, so the target only measures visible defect extent. The evaluation set (test) is only 80 images which is very small and can make findings noisy. Possible data leakage was inconclusive so can't be ruled out and predictions are sensitive to blur or darkening.

Inputs, Outputs and Data

Top Level Inputs: The input data accepted by the solution is a RGB photograph of a pork rasher tray in packaging. The expected formatting for these photos is a file type of either .jpg, .jpeg or .png with a size of exactly 720 x 540 pixels (any other image sizes are rejected as the model was trained with only this size and a size that differs changes the features). The dataset also includes labels for these photographs, but they are not required at prediction time.

Top Level Outputs

Outputs	Type and Units	Meaning and Intended Use
image	string	Path of the image assessed (for traceability).
predicted_area_fraction	float, 0 to 1, dimensionless (0.05 means 5% of the image area)	Estimated total area the human label would box as defective. A severity proxy for ranking trays or feeding decision rule. Clipped at 0 because area cannot be negative.
decision	"pass" or "fail"	“fail” if the predicted area exceeds the threshold. Not a certified result just a screening flag.
threshold	float, same units as the area	Cut off (default 0.04, chosen on validation split by balanced accuracy). Recorded so decisions are reproducible.

Table 1. Top Level Outputs, Meaning and Intended Use

Invalid, Missing or Unsupported Inputs

Problem	Behaviour
File does not exist	FileNotFoundError
Unsupported extension	ValueError then listing the allowed types
Corrupt, truncated or non-image file	ValueError (Full decode is forced, truncation is caught)
Wrong size (not 720 x 540)	ValueError then stating actual/expected size
Missing model file	FileNotFoundError from load_model

Table 2. Behaviour of Invalid, Missing or Unsupported Inputs

Dataset Details: Roboflow "Pork Rasher Error (Packaging)" v4, CC BY 4.0, 3,549 images (3,349 / 120 / 80), 720 × 540, five classes, YOLO box labels. **[1]**

Preprocessing: Auto-orient, resize with fill (centre crop), augmentation (3 versions per source image, exposure ±10% per the README).

Target Construction: For each image, the target is the sum over boxes of width × height, using the normalised YOLO coordinates. An empty label file gives 0.

Computational Approach

The computational workflow is the solution checks the image, extracts 48 features, standardises them, predicts the defect area fraction with a support vector regression model (SVR), clips at 0 and

compares with a threshold of 0.04. Reproducible from fresh clone: scripts/make_filtered_train.py, extract_features.py and train_final_model.py rebuild the model from the Roboflow data, and assess_cli.py scores an image.

How the Method Works:

- **Features:** The presence of defects change the colour of a tray and adds sharp brightness changes such as folds and gaps in the seal. To capture both is three normalised HSV histograms (16 hue, 8 saturation and 8 value bins, 32 total). Edge part is the mean gradient magnitude in each cell of a 4 x 4 grid (16 values), gradient uses central finite differences (np.gradient). A total of 48 features.
- **Scaling:** Each feature is standardised to a zero mean and unit variance across training data only.
- **Regression:** An SVR with an RBF kernel fits a smooth curve through scaled features. This ignores errors smaller than ε and penalises the larger ones. Penalty weight is set by C.
- **Decision:** Clipped prediction is compared to threshold (0.04).

Why was it Selected: The SVR had the best validation R^2 of the three trained models (0.706 for SVR against 0.538 for linear regression and 0.406 for the decision tree) and also the best test result (0.673). These features were chosen because they were cheap to compute (around 20 ms per image), easy to explain and to test as they match methods present in the unit. I chose regression of severity score rather than object detection because a pass/fail packaging decision needs to know the extent of a defect rather then where the defect is.

Assumptions:

- The defect area changes colour and edge properties in a way that is learnable for a smooth function.
- The test and training images have the same conditions (lighting, focus, 720 x 540 size).
- The annotated box label is usable as a true defect area.
- Training, validation and test images are independent (Tried to mitigate leakage but could still be present).

Settings and Parameters

Setting	Value	How it was chosen
Histogram bins	16 / 8 / 8	Fixed
Edge grid	4 x 4	Fixed
SVR kernel	RBF	Fixed
SVR C	1	Validation Grid **[A2]**, 0.1 to 100
SVR ε	0.03	Validation Grid **[A2]**, 0.001 to 0.03
Threshold	0.04	Best balance accuracy on valid
*All other settings are scikit-learn defaults, A2 (Appendix)		

Table 3. Settings and Parameters Table

Implementation: The code is a Python package, meatqa, in src/, one script per experiment in scripts/. It uses NumPy for arrays, Pillow for images and scikit-learn for scaler and models. Results of RMSE, MAE and R^2 are written by hand and tested against hand computed answers. assess_image checks the file type, readability and size before doing any work so an invalid input will fail and display a clear error. All functions have a docstring and pdoc generates the HTML documentation in docs/. There are 39 tests (pytest), a pinned requirements.txt and a MIT license.

Known Limitations: Histograms discard when colour occurs in the image and the 4 x 4 grid is only a coarse position cue. Edge strength depends on both brightness and focus so the model can then fail under darkening and blur. Model gives an amount for the defect not its location and the SVR is harder to interpret than the other simpler models as its settings were tuned on a small validation set (120 images).

Alternatives and Comparison

I compared a baseline and three other trained models on the same features and splits. SVR, Tree depth and final model choice used validation split only. Test split was scored once afterwards.

Model	Test R^2 (Interval)	Test RMSE	Fit (s)	Predict (ms/image)	Stored Model Size	Interpretability
Mean Reference	-0.173	0.0614	0	0	1 number	Trivial
Linear Regression	0.570 (0.380 to 0.694)	0.0372	0.00	0.001	49 numbers	High: One weight per feature
Decision Tree, Depth 6	0.395	0.0441	0.04	0.001	127 nodes at most	High: Readable splits
SVR, RBF	0.673 (0.511 to 0.780)	0.0324	0.40	0.13	3,313 x 48 values at most (~1.3 MB)	Low: Kernel model

Table 4. Alternatives Comparison (Test Split; Interval is a 95% bootstrap interval on R^2)

Accuracy: Every trained model beats the mean reference which scored R^2 below a zero due to the valid and test targets being higher than the training mean. SVR was better on both validation (R^2 0.706) and test. Test RMSE is 47% lower than the reference and 13% lower than the linear model. The paired bootstrap difference between the SVR and linear model has a 95% interval of 0.009 to 0.218, the SVR was better in 98% of resamples hence why the SVR is probably the better option out of these models.

Drawbacks of Models:

- Linear regression is a strong simple baseline (R^2 0.570). It cannot capture the extra non-linear structure the SVR finds.
- The decision tree overfits. Training R^2 rises from 0.365 at depth 2 to 0.989 at depth 15 whilst validation R^2 peaks at 0.406 at depth 6 and then falls.

- The SVR is sensitive to C. Raising C from 1 to 100 lowers validation R^2 from 0.706 to 0.661 or less which is another sign of overfitting. The best setting was C = 1 and ε = 0.03.

Computational Time, Memory and Scalability: SVR is the slowest to fit and predict but 0.13 ms/image is small compared to the 20-26 ms needed to extract features, so speed didn't factor into the choice. Memory is small for all models. Training a kernel SVR typically grows faster than linearly while linear regression and decision trees scale close to linearly so a larger dataset would suit the simpler models but with the dataset having 3,313 images it is small enough not to be a constraint.

Robustness: I only ran brightness and blur tests on the final SVR so I can't confirm if it was more robust then the alternatives.

Suitability: Application needs an accurate severity estimate quickly. It doesn't need an explanation for each decision which is why I accepted a lower interpretability for the best accuracy.

Validation (Method and Results)

What was Validated: How accurate each model predicts the defect area fraction of images it has not yet seen before and how well the threshold of 0.04 separates a pass from a failure.

Method and why it is appropriate: Every image in the dataset came with a ground truth label so I used a three-way split: training data to fit, validation data to choose settings and test data for a final score. Withholding data is appropriate because a model can fit its training images almost perfectly without generalising (seen in depth 15 tree). To prevent image leakage between splits I found source photos by filename (13) and removed all 36 training images involved meaning 3,313 remained. One valid and test overlap remained.

Metrics and Acceptance Criteria: RMSE and MAE are in target units (fraction of image area). R^2 compares a model with the mean reference where 0 means no better and 1 is perfect. A model had to beat the mean reference on test, and the final model had to beat the linear baseline by an interval excluding zero. For the pass/fail I used balanced accuracy (the mean of pass recall and fail recall) due to only 12% of validation pork rasher trays passing so plain accuracy rewards failing everything.

Did Validation Influence Model Selection? Yes, the tree depth, SVR settings, the final model and threshold were all chosen because of validation. For this reason, the test split was kept for one final score. **Appendix A2**

Uncertainty: With 80 test images I bootstrapped the test split (2,000 resamples) to get intervals. The SVR's test R^2 interval is 0.511 to 0.780 which is wide.

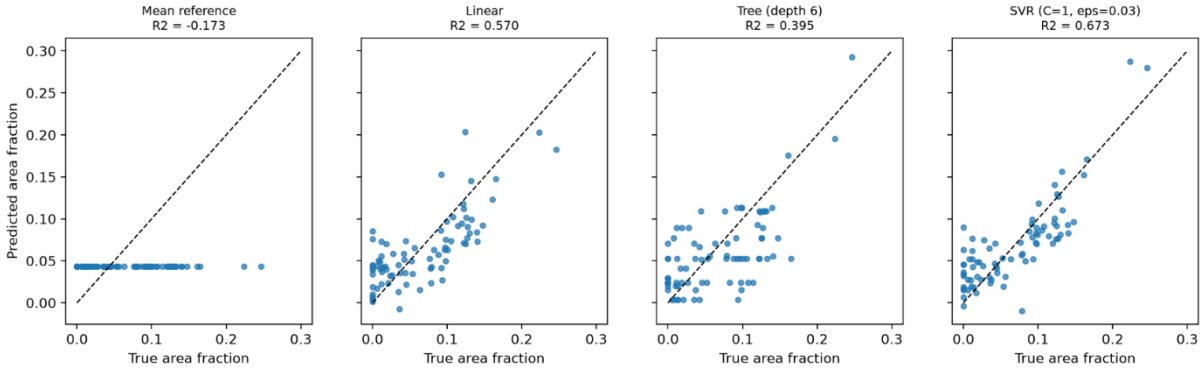


Figure 1. Predicted against true defect area fraction on test split for SVR.

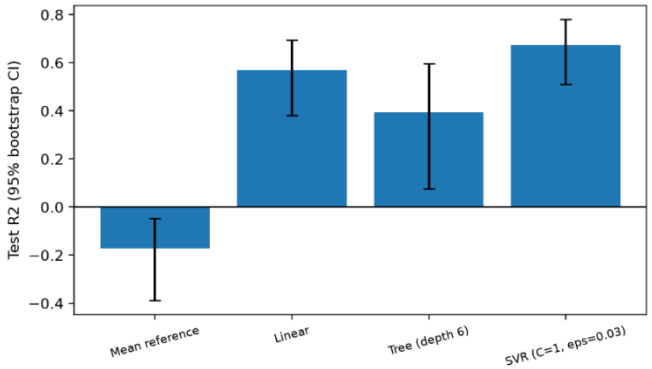


Figure 2. Bootstrap Distribution of Test R^2 for the Linear Model and SVR (Distributions overlap so ranking is likely but not certain).

Error Behaviour: The mean error is almost zero (+0.0012) but not uniform. Trays with a true area above 0.1 (23 images) are under estimated by 0.0161 on average and the 12 defect free test trays are over estimated by 0.0333.

Pass/Fail Condition: On validation the balanced accuracy peaks at a threshold of 0.04 (pass recall 0.857 and fail recall 0.811).

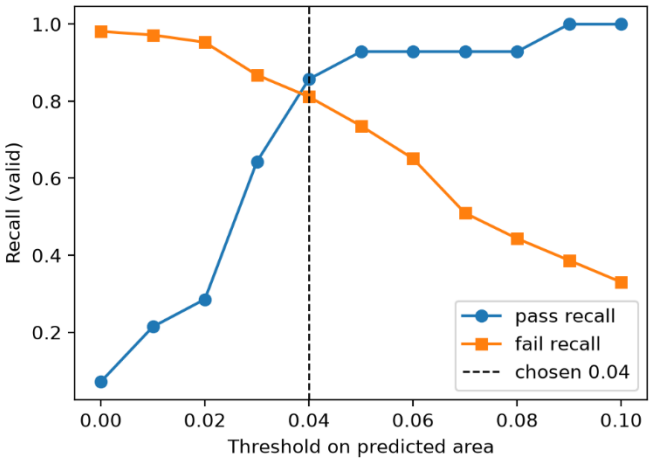


Figure 3. Pass/Fail Recall on Validation Split against Threshold

True Value	Predicted Fail	Predicted Pass
True Fail (68)	53	15
True Pass (12)	4	8

Table 5. Test Result at Threshold 0.04 (Fail is +ve)

A fail recall is 0.779, pass recall is 0.667 and the balanced accuracy is 0.723. Even though failing every tray scores a higher accuracy (0.850) it passes none of the 12 good rasher trays. The weakness lies in the 15 defective trays that pass in a real production line scenario they should lower this threshold and accept more false alarm fail trays for safety and quality assurance.

Sensitivity and Robustness: I changed brightness and blur on test images and re-scored the trained SVR.

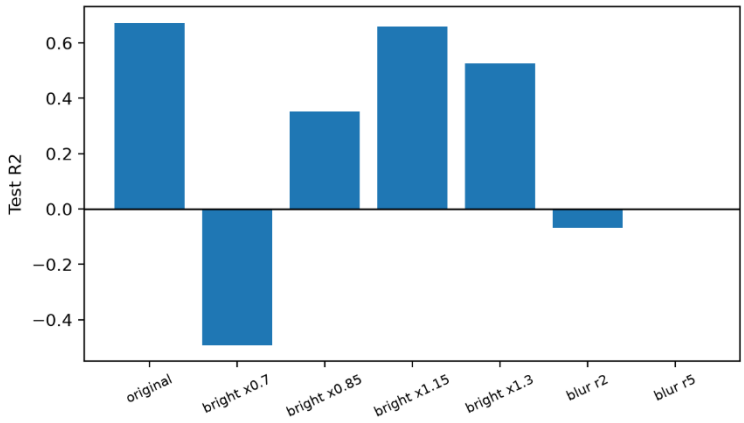


Figure 4. Test R^2 Under Brightness and Blur Changes

Brightening the images by 15% leaves R^2 at 0.660 (Originally 0.673). Darkening to 85% gives 0.352 and to 70% gives -0.492 with mean prediction rising to 0.108 against a true mean of 0.067. Introducing blur gives a R^2 of -0.067 (radius 2) and -0.002 (radius 5). The training images only have ±10% exposure changes so the model should only be used with consistent lighting and focus.

Verification of Code: 39 unit tests use answers worked out by hand. E.g. a flat colour image must give zero edge strength which each histogram must sum to 1. Metric functions checked against hand computed answers. This tests that the code does what I intended and the validation above tests whether my method works.

What Validation Cannot Establish:

- Test set has only 80 images and the left over valid/test leakage overlap may flatter it slightly.
- Valid and test targets differ from training. Score describes dataset rather than production lines in general.
- The ground truth was a box area so a high R^2 shows agreement only with the annotator's boxes from the original Roboflow dataset.
- Wrinkle defect has no test examples so nothing can be claimed about it.

Optimisation and Performance

Fitting and predicting costs milliseconds or less so a feature extraction dominates the runtime and this is what I aimed to optimise.

Profiling: I profiled the extractor on 60 images with cProfile. Profile gave 33 ms per image of which the histograms took 38%, the PIL colour conversions took 22% and the gradient took 16%. Over the full training set, extraction took 26 ms per image (86.6s for 3,313 images).

Version	Time (ms/image)	Test R^2	Effect
Original Extractor (Baseline)	28.1	0.673	Reference
Faster Extractor (Selected)	**19.9**	**0.673**	**About 29% Faster, Identical Output**
Full Size Input (720 x 540)	23.9	0.654	Reference for next two rows
Half Size (360 x 270)	23.9	0.635	No Time Saving and Lower Accuracy
Quarter Size (180 x 135)	14.0	0.601	41% Faster and R^2 Lower by 0.053

Table 6. Baseline Against Improved Versions

The first two rows are the best of three passes over 80 images. The resolution rows come from a quick experiment trained on every third training image so R^2 values differ from the final model. Timings varied between 1 to 3 ms between runs.

Input Resolution and Trade-Offs: Reducing the resolution trades accuracy for speed. Halving the size provides no time saving because the resizing costs around the same as it saves and doing the quarter size saves 10 ms but costs 0.053 in R^2. The faster extractor is seemingly a free gain and with unit tests confirming its output matches the original. I kept full resolution.

Arithmetic Cost and Memory: One image has 388,800 pixels. The gradient needs around 10 operations per pixel (about 4 MFLOP) and the histograms about 1.2 M binning operations which is around 5 MFLOP per image. At 26 ms that is only 0.2 GFLOP/s, far below what a CPU can do, and the arithmetic intensity is low. A few operations per 8-byte value. Code is limited by memory traffic and array handling not the arithmetic which fits the profile. The SVR prediction costs about 3 x 48 operations per support vector (0.5 MFLOP at most). Peak memory is about 22 MB for the float64 image arrays, and the full matrix feature is 1.3 MB.

Numerical Stability: Each histogram is divided by its own sum which is never zero due to every pixel falling in some bin. Features are standardised before the SVR, so no feature dominates the kernel distance. All arrays are float64 and clipping predictions at 0 changes test R^2 only from 0.6732 to 0.6796.

Repeatability and Data Requirements: The scripts gave identical scores when I reran them a whole week apart (SVR validation R^2 and test RMSE) and only the timings varied. This model needs 3,313 training images to fit in under a half second and a larger set would favour those simpler models (linear regression and decision tree).

Final Design: The final selected design is the faster extractor at full resolution with a standard scaler fitted on training data. An SVR with C = 1 and $\varepsilon = 0.03$, clipping at 0 and a 0.04 threshold.

Comparing this to the baseline's it is more accurate and around 29% faster to run at about 20 ms per image end to end. The main drawback and cost of the final selected design is a lower interpretability and its fragility to lighting/blur.

Limitations and Lessons Learned

Limitations:

- With only 80 images the test set is quite small, the SVR's test R^2 interval is 0.511 to 0.780 and its lead over the linear model is just clear of zero.
- Leakage cannot be 100% removed. Thirteen source photos were shared across splits. I excluded the 36 training images involved, but one valid/test overlap remains. My similarity score couldn't determine near duplicates so some more could be undetected.
- There was a distribution shift between splits as the mean defect area in the train set was 0.043, 0.084 for valid and 0.067 for test, the threshold was chosen on valid.
- Missed defects. At the 0.04 threshold 15 of 68 defective test trays pass and large defects are under estimated by 0.016 on average.
- Fragility to lighting and blur. Test R^2 falls to -0.492 at 70% brightness and to around 0 under blur conditions. Training included only ±10% exposure changes.
- The target is a box area which over states true defect pixels, so the model learns agreement with the annotator's boxes.
- Wrinkle has no test examples.
- No localisation of where defects are on the trays just their severity based on the area fraction.

Lessons Learned:

- Compare with baselines first. The mean reference scored R^2 below zero which exposed the shift between splits and set the scale for every score later.
- Check for leakage before modelling. Augmented copies of one photo look alike so splitting by image alone would inflate scores.
- Single score hides failure modes. Headline R^2 looked good but the error analysis and robustness tests show clean trays being flagged and not coping with darkening and blur.
- Validation must be able to say no. Decision tree scored R^2 0.989 with training data at depth = 15 and only 0.287 on validation. Plain accuracy rewarded failing every tray. **[A3]**

Proposed Improvements:

- Train with a wider range of brightnesses and blurriness.
- Split and cross check source photos so leakage is removed by design and collect a larger test set.
- Lower the threshold to reduce risk of passing defective trays.
- If location would be taken into account add spatial colour features.

Conclusion

The model ranks trays by defect severity reasonably well (test R^2 0.673, RMSE 0.0324) but it is not accurate enough to be the only pass/fail gate. Especially when it comes to contexts such as meat production lines and customer safety.

AI-Use Reflection

Tool: Claude (Sonnet 5.5)

What AI was used for: AI was used to assist in drafting Python code, README, tests and docstrings. AI was used as a screening tool to compare written report sections with the rubric and brief to provide advice on what to add, mistakes I've made or what key information is absent. AI helped with providing ideas for me to independently pick from and justify myself.

AI Contribution: I estimate the AI wrote around half of the code and helped structure the documentation. I made every important decision as I wanted to focus on stuff taught directly in the unit. I ran every script and made every commit whilst also writing and editing the report with some AI aid. All numbers and results in this report were my own runs of my GitHub.

Why AI was used: The AI assistance helped let me work through a coherent workflow and help me keep a checklist of where I am at in respect to the deadline. It also helped explain steps I was unsure of to both consolidate and build on my knowledge from this unit. With juggling other units and tests in this unit AI helped with tedious time-consuming tasks such as the coding.

How AI was Checked: Ran each script and compared outputs with the text describing it. Test use answers worked out by hand (E.g. Histogram sums to 1). Installed and checked the fresh clone of the repository. Claims and explanations were checked by sources and the MMA3001 Colab notes.

My Important Decisions: Regression of defect area fraction and not using object detection, excluding the 36 training leakage images, choosing the final model and threshold from validation results. Features, validation, regression models and targets were all chosen by me.

AI Errors and Limitations: Made an error by stating it had 36 features instead of 48. At first, it overstated the near duplicate leakage that I had to go verify and change. Made unneeded scripts and duplicates. Had a local editable install line that would break if cloned by a marker. Limitations would be that AI frequently makes mistakes and trusting AI's plausible answers but when checked with sources it isn't entirely accurate.

Effect on Workflow and Understanding: Provided step by step explanations and gave a linear to do list workflow for a more efficient use of time. E.g. I asked why R^2 can be negative and how the bootstrap works.

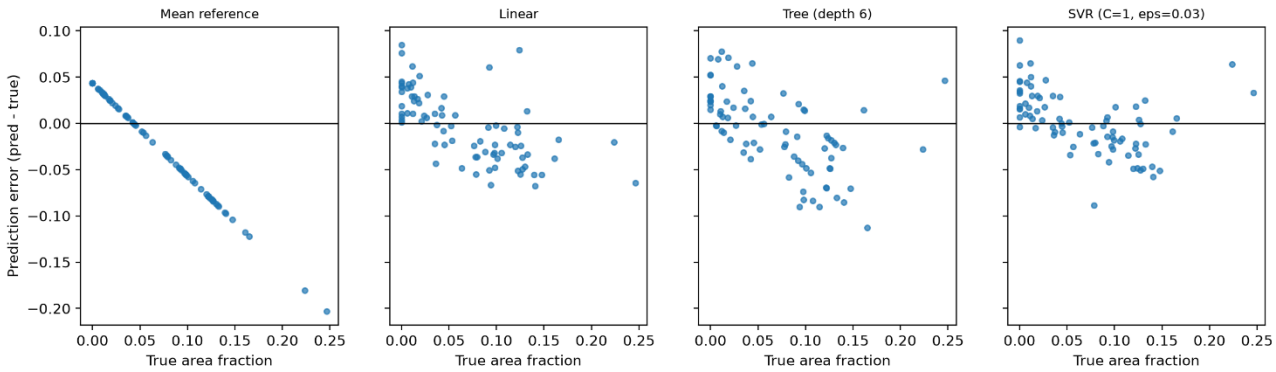
References

1. **Hello. (2025, May 18).** ***Pork rasher error(packaging) dataset*****. Roboflow.** https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging

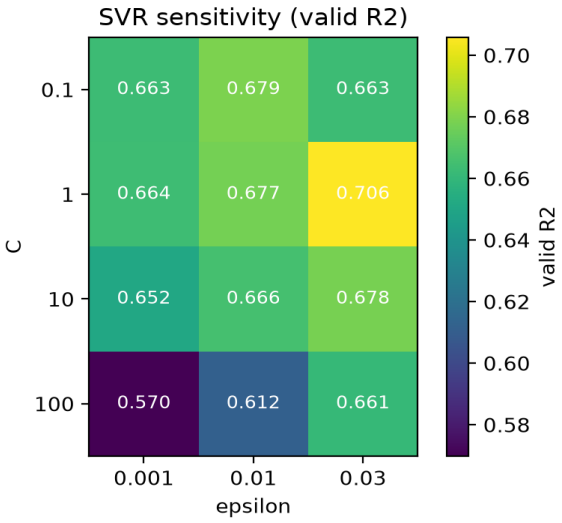
2. **MMA3001 Lecture Notes (Colab)**

3. ***GitHub - rdas0013/MMA3001-meat-quality: Food packaging lines need automated inspection. This tool uses a defect area fraction (total area of an annotated defect box as a share of the image) for a tray photo. The number is then compared with a threshold to give a pass/fail decision. It is a severity proxy and screening aid. It doesn't locate defects.*** **(2026). GitHub.** https://github.com/rdas0013/mma3001-meat-quality

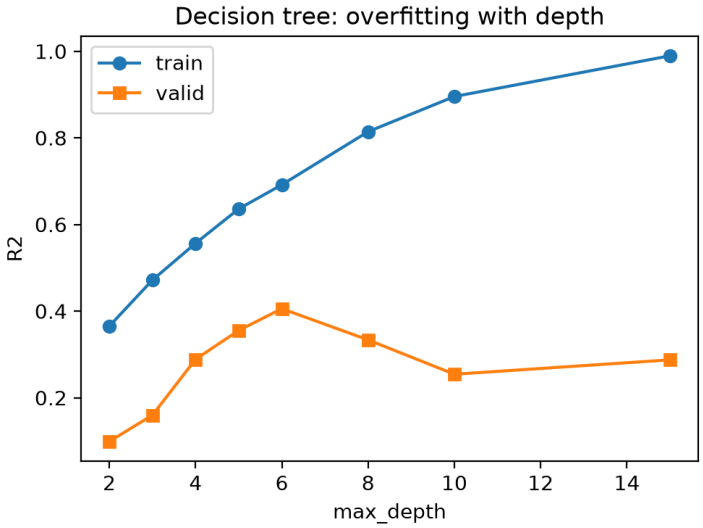
Appendix



A1. Residuals Plot



A2. SVR Hyperparameter Heatmap



A3. Tree Depth Against Validation Error

A4. Repository: https://github.com/rdas0013/mma3001-meat-quality.